

## UNIVERSITY OF CLAUDE · COURSE SYLLABUS

# CSCE 614 — Computer Architecture

*Why your code runs at the speed it does, from the pipeline to the memory hierarchy to the GPU*

**AT A GLANCE**

**Credits:** 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 1 (with CSCE 629 and CSCE 641)

**Prerequisites:** Programming in C or C++; basic digital logic helpful but not assumed

**Effort:** 10–12 hours per week · 13 modules + 2 project weeks

**Deliverable:** A performance portfolio: a set of microbenchmarks that measure your own machine's cache hierarchy, branch predictor, and SIMD width, with every measured number explained

## Course description

CSCE 629 analyses algorithms in a model where every memory access costs one unit and every instruction takes one step. That model is indispensable and it is also wrong by one to two orders of magnitude. This course is about the machine the model abstracts away.

The organising fact is that processors have become very fast and memory has not. A modern core can execute several instructions per cycle; a main-memory access costs two to three hundred cycles. Essentially every architectural feature of the last forty years — caches, prefetchers, out-of-order execution, speculation, wide SIMD, many-threaded GPUs — exists to hide that gap or to find work to do while waiting.

The course is built around measurement. Every mechanism is introduced, then measured on your own hardware with a benchmark you write. You will determine your machine's cache sizes without looking them up, observe the cost of a branch misprediction directly, and watch a loop get eight times faster when it vectorises. A number you measured and explained is worth more than a chapter you read.

The aim is a working cost model: the ability to look at a loop and predict, roughly, what will make it slow. That skill transfers directly to CSCE 641, where frame time is decided by memory access patterns, and to CSCE 735, where it decides whether a parallel algorithm scales.

## Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Measure performance correctly and explain why common methodologies mislead.

- 2 Explain pipelining, the three hazard classes, and how each is resolved.
- 3 Explain branch prediction and quantify the cost of a misprediction.
- 4 Derive your machine's cache hierarchy from measurement alone.
- 5 Restructure code for locality and predict the improvement before measuring it.
- 6 Explain out-of-order execution and what limits instruction-level parallelism.
- 7 Explain why GPUs are throughput machines and what follows for graphics and compute.
- 8 Explain cache coherence, memory consistency, and why lock-free code is difficult.

## Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

| Wk | Module                                                      | Core question it answers                                                        |
|----|-------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1  | <b>Performance: What to Measure</b>                         | What does 'faster' actually mean, and how do you measure it honestly?           |
| 2  | <b>Instruction Sets and the Hardware/Software Interface</b> | What does the hardware actually promise, and what is free to change underneath? |
| 3  | <b>Pipelining and Hazards</b>                               | How do you execute several instructions at once without breaking the program?   |
| 4  | <b>Branch Prediction and Speculation</b>                    | How does a processor keep a deep pipeline full across a branch?                 |
| 5  | <b>Caches: The Memory Wall</b>                              | Why is memory 250 times slower than the processor, and what is done about it?   |
| 6  | <b>Cache Optimization and Prefetching</b>                   | How do you get a 10× speedup without changing the algorithm?                    |
| 7  | <b>Virtual Memory and the TLB</b>                           | What happens between your pointer and a physical address?                       |
| 8  | <b>Out-of-Order Execution</b>                               | How does a processor keep working during a 250-cycle cache miss?                |
| 9  | <b>SIMD and Data-Level Parallelism</b>                      | How do you get 8× throughput without more cores?                                |
| 10 | <b>GPUs as Throughput Machines</b>                          | Why is a GPU fast at graphics and bad at pointer chasing?                       |
| 11 | <b>Multicore and Cache Coherence</b>                        | How do several cores agree on what memory contains?                             |
| 12 | <b>Memory Consistency and Synchronization</b>               | In what order do other cores see your memory operations?                        |
| 13 | <b>Energy, Scaling, and Specialization</b>                  | Why did clock speeds stop rising, and where is performance coming from now?     |

## Projects

---

### Project 1 — Measure your own machine (after Module 7)

Determine your machine's microarchitectural parameters by measurement alone, without consulting a datasheet. Then look them up and account for every discrepancy.

#### Requirements

- Cache sizes at every level, derived from a pointer-chase latency curve against working-set size.
- Cache line size, derived from a strided access experiment.
- Cache associativity, derived from a conflict-miss experiment.
- TLB reach, derived from a page-stride experiment.
- Branch misprediction penalty, in cycles, derived from a random-versus-predictable branch benchmark.
- Memory bandwidth, sequential and random.

#### Done means

- A plot of access latency against working-set size with the cache levels visibly marked and labelled.
- A table comparing every measured value against the manufacturer's specification.
- A written explanation of each discrepancy. 'The prefetcher hid it' is a valid explanation; 'measurement noise' usually is not.

### Project 2 — Make one loop ten times faster (after Module 12)

Take a memory-bound kernel and optimise it by a factor of at least five, using only transformations justified by this course. Every step must be predicted before it is measured.

#### Requirements

- Choose a kernel: dense matrix multiply, a stencil, an N-body step, or a histogram.
- A baseline implementation, honestly written.
- At least four optimisation steps, each with a *prediction* of the speedup written before measuring, and the measured result after.
- Techniques should include at minimum: loop interchange or blocking for locality, a data layout change (AoS to SoA), vectorisation, and one further transformation of your choice.
- Hardware counter data (cache misses, IPC) at each step.

#### Done means

- At least 5× end-to-end speedup, with the final version still producing identical output.
- A table of predicted versus measured speedup for each step.
- An honest account of at least one prediction that was wrong, and why.

## Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

| Component        | Weight | Standard                                                                                        |
|------------------|--------|-------------------------------------------------------------------------------------------------|
| Module exercises | 40%    | Code runs and produces the specified output. Not 'I understand how I would do it.'              |
| Projects         | 45%    | In the portfolio repository, with a README showing results and a written note on what was hard. |
| Self-checks      | 15%    | Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.  |

## Required materials

All free. Nothing in this course requires a purchase.

| Resource                                                                   | Role                                                                                                                               |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| ETH Zürich — Onur Mutlu, Computer Architecture (full lecture videos, free) | Primary lecture series. Graduate level, exceptionally thorough, and freely available in full with slides and assignments.          |
| MIT 6.004 Computation Structures (OCW, Spring 2017)                        | The foundation course: from transistors to a working pipelined processor. Start here if digital logic is unfamiliar.               |
| MITx 6.004.1x Computation Structures: Digital Circuits (edX)               | The same material in graded MOOC form, free to audit.                                                                              |
| Nand2Tetris — build a computer from NAND gates (free)                      | If you have never built a processor, do the first six projects. The understanding it produces is disproportionate to the effort.   |
| Ulrich Drepper — What Every Programmer Should Know About Memory (free)     | The definitive practical treatment of the memory hierarchy. Long, dense, and worth every page. Core reading for Modules 05–07.     |
| Agner Fog — Software optimization manuals (free)                           | Microarchitecture tables, instruction latencies, and optimisation guidance for every recent x86 core. The reference you will keep. |

## Tooling

- **C or C++ with a good compiler.** You need to see generated assembly and control optimisation flags.
- **Compiler Explorer (godbolt.org)** — the fastest way to see what your source becomes. Used constantly from Module 02.

- **perf** (Linux) or **Intel VTune** / **AMD uProf** (Windows) for hardware performance counters: cache misses, branch mispredictions, IPC.
- A **timing harness** with proper warm-up, repeated trials, and median reporting — reuse the one from CSCE 629 Module 01.
- A **portfolio repository** with one directory per module, each containing the benchmark, the measured data, and a written explanation.

## Academic honesty, restated for a solo student

---

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.